

Integration Manual

for S32K3XX CANTRCV_TJA1145A Driver

Document Number: IM34CANTRCVTJA1145AASR4.7 Rev0000R0.8.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	5
2.4 Acronyms and Definitions	6
2.5 Reference List	6
3 Building the driver	7
3.1 Build Options	7
3.1.1 GCC Compiler/Assembler/Linker Options	8
3.1.2 DIAB Compiler/Assembler/Linker Options	10
3.1.3 GHS Compiler/Assembler/Linker Options	12
3.1.4 IAR Compiler/Assembler/Linker Options	14
3.2 Files required for compilation	16
3.3 Setting up the plugins	16
4 Function calls to module	17
4.1 Function Calls during Start-up	17
4.2 Function Calls during Shutdown	17
4.3 Function Calls during Wake-up	17
5 Module requirements	18
5.1 Exclusive areas to be defined in BSW scheduler	18
5.2 Peripheral Hardware Requirements	19
5.3 ISR to configure within AutosarOS - dependencies	19
5.4 ISR Macro	19
5.4.1 Without an Operating System	19
5.4.2 With an Operating System	20
5.5 Other AUTOSAR modules - dependencies	20
5.6 Data Cache Restrictions	20
5.7 User Mode support	20
5.7.1 User Mode configuration in AutosarOS	21
5.8 Multicore support	21
6 Main API Requirements	22
6.1 Main function calls within BSW scheduler	22
6.2 API Requirements	22
6.3 Calls to Notification Functions, Callbacks, Callouts	22
7 Memory allocation	23

7.1 Sections to be defined in CanTrcv_43_tja1145a_MemMap.h	23
7.2 Linker command file	23
8 Integration Steps	24
9 External assumptions for driver	25

Chapter 1

Revision History

Revision	Date	Author	Description
1.0	May 22 2023	NXP AASW Team	CANTRCV TJA1145A AUTOSAR R21-11 Version 0.8.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This integration manual describes the integration requirements for CanTrcv Driver.

The CanTrcv driver is intended to control TJA1145A CAN transceiver.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32k310_mqfp100
- s32k310_lqfp48
- s32k311_mqfp100 / MWCT2015S_mqfp100
- s32k311_lqfp48
- s32k312_mqfp100 / MWCT2016S_mqfp100
- s32k312_mqfp172 / MWCT2016S_mqfp172
- s32k314_mqfp172
- s32k314_mapbga257
- s32k322_mqfp100 / MWCT2D16S_mqfp100
- s32k322_mqfp172 / MWCT2D16S_mqfp172

- s32k324_mqfp172 / MWCT2D17S_mqfp172
- s32k324_mapbga257
- s32k341_mqfp100
- s32k341_mqfp172
- s32k342_mqfp100
- s32k342_mqfp172
- s32k344_mqfp172
- s32k344_mapbga257
- s32k394_mapbga289
- s32k396_mapbga289
- s32k358_mqfp172
- s32k358_mapbga289
- s32k328_mqfp172
- s32k328_mapbga289
- s32k338_mqfp172
- s32k338_mapbga289
- s32k348_mqfp172
- s32k348_mapbga289
- s32m274_lqfp64
- s32m276_lqfp64

All of the above microcontroller devices are collectively named as S32K3.

Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
ASM	Assembler
BSMI	Basic Software Make file Interface
CAN	Controller Area Network
C/CPP	C and C++ Source Code
CS	Chip Select
CTU	Cross Trigger Unit
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DMA	Direct Memory Access
ECU	Electronic Control Unit
FIFO	First In First Out
LSB	Least Significant Bit
MCU	Micro Controller Unit
MIDE	Multi Integrated Development Environment
MSB	Most Significant Bit
N/A	Not Applicable
RAM	Random Access Memory
SIU	Systems Integration Unit
SWS	Software Specification
VLE	Variable Length Encoding
XML	Extensible Markup Language

2.5 Reference List

#	Title	Version
1	Specification of CanTrcv Driver	AUTOSAR Release R21-11
2	TJA1145A objective datasheet	2

Chapter 3

Building the driver

- [Build Options](#)
- [Files required for compilation](#)
- [Setting up the plugins](#)

This section describes the source files and various compilers, linker options used for building the driver. It also explains the EB Tresos Studio plugin setup procedure.

3.1 Build Options

- [GCC Compiler/Assembler/Linker Options](#)
- [DIAB Compiler/Assembler/Linker Options](#)
- [GHS Compiler/Assembler/Linker Options](#)
- [IAR Compiler/Assembler/Linker Options](#)

The RTD driver files are compiled using:

- NXP GCC 10.2.0 20200723 (Build 1728 Revision g5963bc8)
- Wind River Diab Compiler 7.0.4
- Compiler Versions: Green Hills Multi 7.1.6d / Compiler 2021.1.4
- Compiler Versions: IAR ANSI C/C++ Compiler V8.50.10 (safety version)

The compiler, assembler, and linker flags used for building the driver are explained below.

The TS_T40D34M30I0R0 part of the plugin name is composed as follows:

- T = Target_Id (e.g. T40 identifies Cortex-M architecture)
- D = Derivative_Id (e.g. D34 identifies S32K3 platform)
- M = SW_Version_Major and SW_Version_Minor
- I = SW_Version_Patch
- R = Reserved

3.1.1 GCC Compiler/Assembler/Linker Options

3.1.1.1 GCC Compiler Options

Compiler Option	Description
-mcpu=cortex-m7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mlittle-endian	Generate code for a processor running in little-endian mode
-mfpv=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-std=c99	Specifies the ISO C99 base standard
-Os	Optimize for size. Enables all -O2 optimizations except those that often increase code size
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-Wall	Enables all the warnings about constructions that some users consider questionable, and that are easy to avoid (or modify to prevent the warning), even in conjunction with macros
-Wextra	This enables some extra warning flags that are not enabled by -Wall
-pedantic	Issue all the warnings demanded by strict ISO C. Reject all programs that use forbidden extensions. Follows the version of the ISO C standard specified by the aforementioned -std option
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wundef	Warn if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero
-Wunused	Warn whenever a function, variable, label, value, macro is unused
-Werror=implicit-function-declaration	Make the specified warning into an error. This option throws an error when a function is used before being declared
-Wsign-compare	Warn when a comparison between signed and unsigned values could produce an incorrect result when the signed value is converted to unsigned.
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-fno-short-enums	Specifies that the size of an enumeration type is at least 32 bits regardless of the size of the enumerator values.
-funsigned-char	Let the type char be unsigned by default, when the declaration does not use either signed or unsigned
-funsigned-bitfields	Let a bit-field be unsigned by default, when the declaration does not use either signed or unsigned

Compiler Option	Description
-fno-common	Makes the compiler place uninitialized global variables in the BSS section of the object file. This inhibits the merging of tentative definitions by the linker so you get a multiple-definition error if the same variable is accidentally defined in more than one compilation unit
-fstack-usage	This option is only used to build test for generation Ram/↔ Stack size report. Makes the compiler output stack usage information for the program, on a per-function basis
-fdump-ipa-all	This option is only used to build test for generation Ram/↔ Stack size report. Enables all inter-procedural analysis dumps
-c	Stop after assembly and produce an object file for each source file
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-D \$ (DERIVATIVE)	Predefine S32K3's derivative as a macro, with definition 1. For example: Predefine for S32K344 will be -DS32K344.
-DGCC	Predefine GCC as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.↔ c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.
-sysroot=	Specifies the path to the sysroot, for Cortex-M7 it is /arm-none-eabi/newlib
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.1.2 GCC Assembler Options

Assembler Option	Description
-Xassembler-with-cpp	Specifies the language for the following input files (rather than letting the compiler choose a default based on the file name suffix)
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mthumb	Generates code that executes in Thumb state
-c	Stop after assembly and produce an object file for each source file

3.1.1.3 GCC Linker Options

Linker Option	Description
-Wl,-Map,filename	Produces a map file
-T linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-entry=Reset_Handler	Specifies that the program entry point is Reset_Handler
-nostartfiles	Do not use the standard system startup files when linking
-mcpu=cortexm7	Targeted ARM processor for which GCC should tune the performance of the code
-mthumb	Generates code that executes in Thumb state
-mfpu=fpv5-sp-d16	Specifies the floating-point hardware available on the target
-mfloat-abi=hard	Specifies the floating-point ABI to use. "hard" allows generation of floating-point instructions and uses FPU-specific calling conventions
-mlittle-endian	Generate code for a processor running in little-endian mode
-ggdb3	Produce debugging information for use by GDB using the most expressive format available, including GDB extensions if at all possible. Level 3 includes extra information, such as all the macro definitions present in the program
-lc	Link with the C library
-lm	Link with the Math library
-lgcc	Link with the GCC library
-specs=nano.specs	Use Newlib nano specs
-specs=nosys.specs	Do not use printf/scanf

3.1.2 DIAB Compiler/Assembler/Linker Options

3.1.2.1 DIAB Compiler Options

Compiler Option	Description
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)
-mthumb	Selects generating code that executes in Thumb state
-std=c99	Follows the C99 standard for C
-Oz	Like -O2 with further optimizations to reduce code size
-g	Generates DWARF 4.0 debug information
-fstandalone-debug	Emits full debug info for all types used by the program
-Wstrict-prototypes	Warn if a function is declared or defined without specifying the argument types
-Wsign-compare	Produce warnings when comparing signed type with unsigned type
-Wdouble-promotion	Give a warning when a value of type float is implicitly promoted to double
-Wunknown-pragmas	Issues a warning for unknown pragmas
-Wundef	Warns if an undefined identifier is evaluated in an #if directive. Such identifiers are replaced with zero

Compiler Option	Description
-Wextra	Enables some extra warning flags that are not enabled by '-Wall'
-Wall	Enables all of the most useful warnings (for historical reasons this option does not literally enable all warnings)
-pedantic	Emits a warning whenever the standard specified by the -std option requires a diagnostic
-Werror=implicit-function-declaration	Generates an error whenever a function is used before being declared
-fno-common	Compile common globals like normal definitions
-fno-signed-char	Char is unsigned
-fno-trigraphs	Do not process trigraph sequences
-V	Displays the current version number of the tool suite
-c	Stop after assembly and produce an object file for each source file
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-D \$ (DERIVATIVE)	Predefine S32K3's derivative as a macro, with definition 1
-DDIAB	Predefine DIAB as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.2.2 DIAB Assembler Options

Assembler Option	Description
-mthumb	Selects generating code that executes in Thumb state
-Xpreprocess-assembly	Invokes C preprocessor on assembly files before running the assembler
-Xassembly-listing	Produces an .lst assembly listing file
-c	Stop after assembly and produce an object file for each source file
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)

3.1.2.3 DIAB Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
linker_script_file.dld	Use linker_script_file.dld as the linker script. This script replaces the default linker script (rather than adding to it)
-m30	m2 + m4 + m8 + m16
-Xstack-usage	Gathers and display stack usage at link time
-Xpreprocess-lecl	Perform pre-processing on linker scripts
-Llibrary_path	Points to the libraries location for ARMV7EMMG to be used for linking
-lc	Links with the standard C library
-lm	Links with the math library
-tARMCORTEXM7MG:simple	Selects target processor (hardware single-precision, software double-precision floating-point)

3.1.3 GHS Compiler/Assembler/Linker Options

3.1.3.1 GHS Compiler Options

Compiler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-thumb	Selects generating code that executes in Thumb state
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-C99	Use (strict ISO) C99 standard (without extensions)
-ghstd=last	Use the most recent version of Green Hills Standard mode (which enables warnings and errors that enforce a stricter coding standard than regular C and C++)
-Osize	Optimize for size
-gnu_asm	Enables GNU extended asm syntax support
-dual_debug	Generate DWARF 2.0 debug information
-G	Generate debug information
-keeptempfiles	Prevents the deletion of temporary files after they are used. If an assembly language file is created by the compiler, this option will place it in the current directory instead of the temporary directory
-Wimplicit-int	Produce warnings if functions are assumed to return int
-Wshadow	Produce warnings if variables are shadowed
-Wtrigraphs	Produce warnings if trigraphs are detected
-Wundef	Produce a warning if undefined identifiers are used in #if preprocessor statements
-unsigned_chars	Let the type char be unsigned, like unsigned char
-unsigned_fields	Bitfields declared with an integer type are unsigned

Compiler Option	Description
-no_commons	Allocates uninitialized global variables to a section and initializes them to zero at program startup
-no_exceptions	Disables C++ support for exception handling
-no_slash_comment	C++ style // comments are not accepted and generate errors
-prototype_errors	Controls the treatment of functions referenced or called when no prototype has been provided
-incorrect_pragma_warnings	Controls the treatment of valid #pragma directives that use the wrong syntax
-c	Stop after assembly and produce an object file for each source file
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-D \$ (DERIVATIVE)	Predefine S32K3's derivative as a macro, with definition 1. For example: Predefine for S32K344 will be -DS32K344.
-DGHS	Predefine GHS as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode

3.1.3.2 GHS Assembler Options

Assembler Option	Description
-cpu=cortexm7	Selects target processor: Arm Cortex M7
-fpu=vfpv5_d16	Specifies hardware floating-point using the v5 version of the VFP instruction set, with 16 double-precision floating-point registers
-fsingle	Use hardware single-precision, software double-precision FP instructions
-preprocess_assembly_files	Controls whether assembly files with standard extensions such as .s and .asm are preprocessed
-list	Creates a listing by using the name and directory of the object file with the .lst extension
-c	Stop after assembly and produce an object file for each source file

3.1.3.3 GHS Linker Options

Linker Option	Description
-e Reset_Handler	Make the symbol Reset_Handler be treated as a root symbol and the start label of the application
-T linker_script_file.ld	Use linker_script_file.ld as the linker script. This script replaces the default linker script (rather than adding to it)
-map	Produce a map file
-keepmap	Controls the retention of the map file in the event of a link error
-Mn	Generates a listing of symbols sorted alphabetically/numerically by address
-delete	Instructs the linker to remove functions that are not referenced in the final executable. The linker iterates to find functions that do not have relocations pointing to them and eliminates them
-ignore_debug_references	Ignores relocations from DWARF debug sections when using -delete. DWARF debug information will contain references to deleted functions that may break some third-party debuggers
-Llibrary_path	Points to library_path (the libraries location) for thumb2 to be used for linking
-larch	Link architecture specific library
-lstartup	Link run-time environment startup routines. The source code for themodules in this library is provided in the src/libstartup directory
-lind_sd	Link language-independent library, containing support routines for features such as software floating point, run-time error checking, C99 complex numbers, and some general purpose routines of the ANSI C library
-v	Prints verbose information about the activities of the linker, including the libraries it searches to resolve undefined symbols
-keep=C40_Ip_AccessCode	Avoid linker remove function C40_Ip_AccessCode from Fls module because it is not referenced explicitly
-nostartfiles	Controls the start files to be linked into the executable

3.1.4 IAR Compiler/Assembler/Linker Options

3.1.4.1 IAR Compiler Options

Compiler Option	Description
-cpu Cortex-M7	Targeted ARM processor for which IAR should tune the performance of the code
-cpu_mode thumb	Generates code that executes in Thumb state
-endian little	Generate code for a processor running in little-endian mode
-fpu VFPv5-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant.
-e	Enables all IAR C language extensions
-Ohs	Optimize for size. the compiler will emit AEABI attributes indicating the requested optimization goal. This information can be used by the linker to select smaller or faster variants of DLIB library functions
-debug	Makes the compiler include debugging information in the object modules. Including debug information will make the object files larger

Compiler Option	Description
-no_clustering	Disables static clustering optimizations. Static and global variables defined within the same module will not be arranged so that variables that are accessed in the same function are close to each other
-no_mem_idioms	Makes the compiler not optimize certain memory access patterns
-do_explicit_zero_opt_in_named_sections	Disable the exception for variables in user-named sections, and thus treat explicit initializations to zero as zero initializations, not copy initializations
-require_prototypes	Force the compiler to verify that all functions have proper prototypes. Generates an error otherwise
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages
-diag_suppress Pa050	Suppresses diagnostic message Pa050
-DS32K3XX	Predefine S32K3XX as a macro, with definition 1
-D \$ (DERIVATIVE)	Predefine S32K3's derivative as a macro, with definition 1. For example: Predefine for S32K344 will be -DS32K344.
-DIAR	Predefine IAR as a macro, with definition 1
-DUSE_SW_VECTOR_MODE	Predefine USE_SW_VECTOR_MODE as a macro, with definition 1. By default, the drivers are compiled to handle interrupts in Software Vector Mode.
-DD_CACHE_ENABLE	Predefine D_CACHE_ENABLE as a macro, with definition 1. Enables data cache initialization in source file system.c under the Platform driver
-DI_CACHE_ENABLE	Predefine I_CACHE_ENABLE as a macro, with definition 1. Enables instruction cache initialization in source file system.c under the Platform driver
-DENABLE_FPU	Predefine ENABLE_FPU as a macro, with definition 1. Enables FPU initialization in source file system.c under the Platform driver
-DMCAL_ENABLE_USER_MODE_SUPPORT	Predefine MCAL_ENABLE_USER_MODE_SUPPORT as a macro, with definition 1. Allows drivers to be configured in user mode.

3.1.4.2 IAR Assembler Options

Assembler Option	Description
-cpu Cortex-M7	Targeted ARM processor for which IAR should generate the instruction set
-fpu VFPv5-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant.
-cpu_mode thumb	Selects the thumb mode for the assembler directive CODE
-g	Disables the automatic search for system include files
-r	Generates debug information

3.1.4.3 IAR Linker Options

Linker Option	Description
-map filename	Produces a map file
-config linkerfile	Use linkerfile as the linker script. This script replaces the default linker script (rather than adding to it)
-cpu=Cortex-M7	Selects the ARM processor variant to link the application for
-fpu VFPv5-SP	Use this option to generate code that performs floating-point operations using a Floating Point Unit (FPU). Single-precision variant.
-entry _start	Treats _start as a root symbol and start label
-enable_stack_usage	Enables stack usage analysis. If a linker map file is produced, a stack usage chapter is included in the map file
-skip_dynamic_initialization	Dynamic initialization (typically initialization of C++ objects with static storage duration) will not be performed automatically during application startup
-no_wrap_diagnostics	Does not wrap long lines in diagnostic messages

3.2 Files required for compilation

This section describes the include files required to compile, assemble (if assembler code) and link the CanTrcv driver.

To avoid integration of incompatible files, all the include files from other modules shall have the same AR_MAJOR_VERSION and AR_MINOR_VERSION, i.e. only files with the same AUTOSAR 4.7 (R21-11) versions can be compiled.

The CanTrcv driver needs Can, CanIf, Port, Dem, Det, Mcu and Spi modules for proper operation, see Other AUTOSAR modules dependencies section of the IM.

3.3 Setting up the plugins

The CanTrcv driver was designed to be configured by using the EB Tresos Studio.

For proper operation of the CanTrcv driver configure at least one CanTrcv channel (for which you will need to configure Spi and Port plugins) and create at least one device configuration, which will be used during initialization.

Chapter 4

Function calls to module

- [Function Calls during Start-up](#)
- [Function Calls during Shutdown](#)
- [Function Calls during Wake-up](#)

4.1 Function Calls during Start-up

The CanTrcv Driver shall be initialized through the CanTrcv_Init API function during the UP phase of EcuM (which means this module will be initialized by the BswM to allow for a short STARTUP phase). The Mcu, Spi, Can and Port drivers shall be initialized beforehand.

4.2 Function Calls during Shutdown

None.

4.3 Function Calls during Wake-up

None.

Chapter 5

Module requirements

- Exclusive areas to be defined in BSW scheduler
- unavailable_exclusive_areas
- Peripheral Hardware Requirements
- ISR to configure within AutosarOS - dependencies
- ISR Macro
- Other AUTOSAR modules - dependencies
- Data Cache Restrictions
- User Mode support
- Multicore support

5.1 Exclusive areas to be defined in BSW scheduler

__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_00__	is used in function CanTrcv_43_tja1145a_Init to update registers via T
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_00__	is used in function CanTrcv_43_tja1145a_SetOpMode to update registers
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_00__	is used in function CanTrcv_43_tja1145a_SetPNActivationState to update
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_01__	is used in function CanTrcv_43_tja1145a_GetOpMode to read device mode
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_01__	is used in function CanTrcv_43_tja1145a_SetOpMode to check current mod
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_01__	is used in function CanTrcv_43_tja1145a_DeInit to check current mode t
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__	is used in function CanTrcv_43_tja1145a_GetBusWuReason to get wake up
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__	is used in function CanTrcv_43_tja1145a_SetWakeupMode to check wake up
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__	is used in function CanTrcv_43_tja1145a_CheckWakeup to check wake up r
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__	is used in function CanTrcv_43_tja1145a_CheckWakeFlag to check wake up
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__	is used in function CanTrcv_43_tja1145a_Init to check wake up reason u
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_03__	is used in function CanTrcv_43_tja1145a_ReadTrcvTimeoutFlag to read ti
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_04__	is used in function CanTrcv_43_tja1145a_ReadTrcvSilenceFlag to read si
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_05__	is used in function CanTrcv_43_tja1145a_GetTrcvSystemData to read tran
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_06__	is used in function CanTrcv_43_tja1145a_Init to read POR event status
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_06__	is used in function CanTrcv_43_tja1145a_SetOpMode to read POR event st
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_07__	is used in function CanTrcv_43_tja1145a_Init to read system error stat
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_07__	is used in function CanTrcv_43_tja1145a_SetOpMode to read system error
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_07__	is used in function CanTrcv_43_tja1145a_MainFunctionDiagnostics to rea
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_08__	is used in function CanTrcv_43_tja1145a_Exts_ClearSystemError to clear
__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_08__	is used in function CanTrcv_43_tja1145a_ClearTrcvTimeoutFlag to clear

Exclusive Areas implemented in Low level driver layer (IPL)

__CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_00__ is used in function TJA1145A_DRV_UpdateRegister to ensure uninterrupted
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_01__ is used in function TJA1145A_DRV_GetMode to read Mode Control Register
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_02__ is used in function TJA1145A_DRV_GetWuReason to read and parse status.
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_03__ is used in function TJA1145A_DRV_ReadTimeoutFlag to read timeout flag.
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_04__ is used in function TJA1145A_DRV_ReadSilenceFlag to read silence flag.
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_05__ is used in function TJA1145A_DRV_GetTrcvSystemData to read system data.
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_06__ is used in function TJA1145A_DRV_GetPorEventStatus to read and parse P
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_07__ is used in function TJA1145A_DRV_GetSystemErrorStatus to parse system
 __CanTrcv_tja1145a_CANTRCV_EXCLUSIVE_AREA_08__ is used in function TJA1145A_DRV_ClearSystemErrorStatus to clear system

5.2 Peripheral Hardware Requirements

TJA1145A

5.3 ISR to configure within AutosarOS - dependencies

None.

5.4 ISR Macro

RTD drivers use the ISR macro to define the functions that will process hardware interrupts. Depending on whether the OS is used or not, this macro can have different definitions.

5.4.1 Without an Operating System The macro *USING_OS_AUTOSAROS* must not be defined.

5.4.1.1 Using Software Vector Mode

The macro *USE_SW_VECTOR_MODE* must be defined and the ISR macro is defined as:

```
#define ISR(IsrName) void IsrName(void)
```

In this case, the drivers' interrupt handlers are normal C functions and their prologue/epilogue will handle the context save and restore.

5.4.1.2 Using Hardware Vector Mode

The macro *USE_SW_VECTOR_MODE* must not be defined and the ISR macro is defined as:

```
#define ISR(IsrName) INTERRUPT_FUNC void IsrName(void)
```

In this case, the drivers' interrupt handlers must also handle the context save and restore.

5.4.2 With an Operating System Please refer to your OS documentation for description of the ISR macro.

5.5 Other AUTOSAR modules - dependencies

- **Can:** Needed for configuration of the CanTrcv driver.
- **CanIf:** The callback *CanIf_TrcevModeIndication* is used to notify the upper layer about mode transitions in the controllers.
- **Port:** Needed to configure the pins to be used by the CanTrcv driver.
- **Spi:** Needed to communicate with the CanTrcv driver.
- **Det:** Needed for detecting and reporting development errors. The API function used is *Det_ReportError*. The detection can be configured using the *CanTrcvDevErrorDetect* configuration parameter.
- **Dem:** Needed for processing and storing diagnostic events and data. The API function used is *Dem_SetEventStatus*. The detection can be configured using the *CanTrcvDiagnosticEventManager* configuration parameter.
- **Base:** Needed for common files/definitions which are used by all RTD modules.
- **Mcu:** Needed to configure the clocks to be used by the CanTrcv driver.
- **Platform:** The PLATFORM module is required for interrupt service routines.
- **Resource:** Sub-Derivative model is selected from Resource configuration.
- **EcuC:** Needed to retrieve information about post-build variants.
- **EcuM:** This module is used for processing the Wakeup notifications of CAN. Whenever the module is in Sleep mode and a wakeup event occurs, it is reported to EcuM through the *EcuM_CheckWakeupEvent()* API.
- **Os:** Needed to define a mapping between EcuC partitions and EcuC core ids when multicore support is enabled.

5.6 Data Cache Restrictions

In the DMA transfer mode, DMA transfers may issue cache coherency problems. To avoid possible coherency issues when D-CACHE is enabled, the user shall ensure that the buffers used as TCD source and destination are allocated in the NON-CACHEABLE area (by means of *CanTrcv_43_tja1145a_Memmap*).

5.7 User Mode support

No special measures need to be taken to run CanTrcv module from user mode. The CanTrcv driver code can be executed at any time from both supervisor and user mode.

- *user_mode_config_in_module*
- [User Mode configuration in AutosarOS](#)

5.7.1 User Mode configuration in AutosarOS

When User mode is enabled, the driver may have the functions that need to be called as trusted functions in AutosarOS context. Those functions are already defined in driver and declared in the header `<IpName>_Ip_TrustedFunctions.h`. This header also included all headers files that contains all types definition used by parameters or return types of those functions. Refer the chapter `user_mode_config_in_module` for more detail about those functions and the name of header files they are declared inside. Those functions will be called indirectly with the naming convention below in order to AutosarOS can call them as trusted functions.

`Call_<Function_Name>_TRUSTED(parameter1,parameter2,...)`

That is the result of macro expansion `OsIf_Trusted_Call` in driver code:

```
#define OsIf_Trusted_Call[1-6params](name,param1,...,param6) Call_##name##_TRUSTED(param1,...,param6)
```

So, the following steps need to be done in AutosarOS:

- Ensure `MCAL_ENABLE_USER_MODE_SUPPORT` macro is defined in the build system or somewhere global.
- Define and declare all functions that need to call as trusted functions follow the naming convention above in Integration/User code. They need to be visible in `Os.h` for the driver to call them. They will do the marshalling of the parameters and call `CallTrustedFunction()` in OS specific manner.
- `CallTrustedFunction()` will switch to privileged mode and call `TRUSTED_<Function_Name>()`.
- `TRUSTED_<Function_Name>()` function is also defined and declared in Integration/User code. It will un-marshalling of the parameters to call `<Function_Name>()` of driver. The `<Function_Name>()` functions are already defined in driver and declared in `<IpName>_Ip_TrustedFunctions.h`. This header should be included in OS for OS call and indexing these functions.

See the sequence chart below for an example calling `Linflexd_Uart_Ip_Init_Privileged()` as a trusted function.

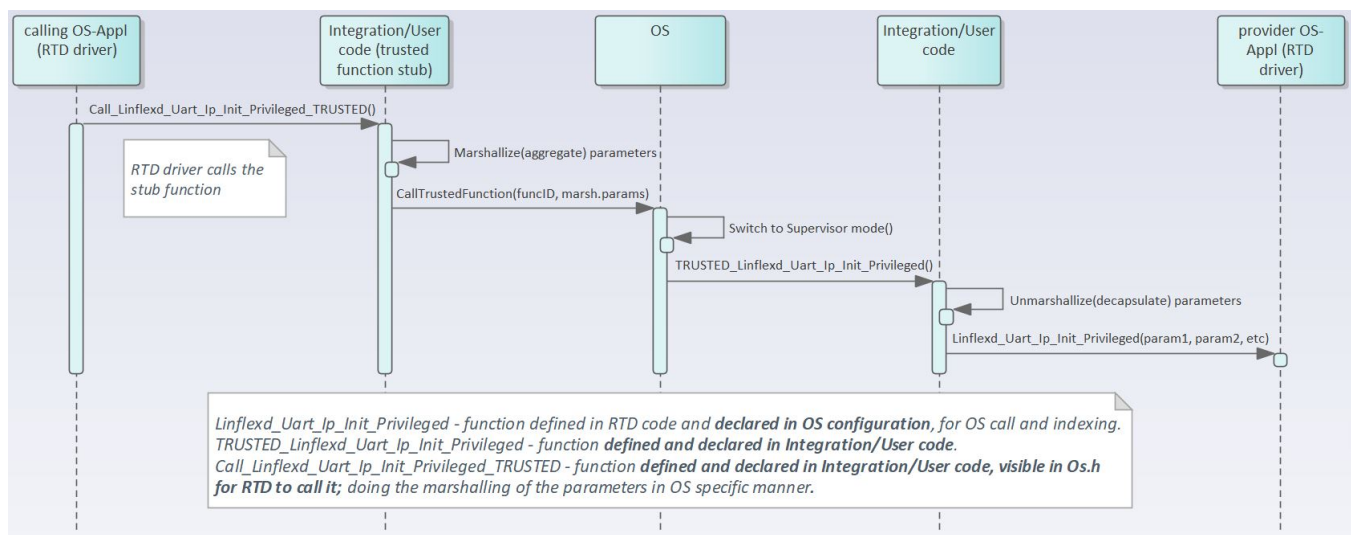


Figure 5.1 Example sequence chart for calling `Linflexd_Uart_Ip_Init_Privileged` as trusted function

5.8 Multicore support

None.



Chapter 6

Main API Requirements

- [Main function calls within BSW scheduler](#)
- [API Requirements](#)
- [Calls to Notification Functions, Callbacks, Callouts](#)

6.1 Main function calls within BSW scheduler

None.

6.2 API Requirements

None.

6.3 Calls to Notification Functions, Callbacks, Callouts

The CanTrcv Driver provides no configurable notification functions, callbacks, or callouts.

Chapter 7

Memory allocation

- [Sections to be defined in CanTrcv_43_tja1145a_MemMap.h](#)
- [Linker command file](#)

7.1 Sections to be defined in CanTrcv_43_tja1145a_MemMap.h

Section name	Type of section	Description
CANTRCV_43_TJA1145A_↔ START_SEC_CONFIG_DATA_↔ UNSPECIFIED	Configuration Data	Start of Memory Section for Config Data
CANTRCV_43_TJA1145A_START↔ _SEC_CODE	Code	Start of memory Section for Code
CANTRCV_43_TJA1145A_START↔ _SEC_CONST_UNSPECIFIED	Constant Data	The parameters that are not variant aware shall be stored in memory section for constants.
CANTRCV_43_TJA1145A_STOP_↔ SEC_VAR_INIT_BOOLEAN	Variables	End of above section.
CANTRCV_43_TJA1145A_↔ START_SEC_VAR_CLEARED_↔ UNSPECIFIED_NO_CACHEABLE	Non-Cacheable Variables	Used for variables, structures, arrays when the SIZE (alignment) does not fit the criteria of 8,16 or 32 bit, and that have to be stored in a non-cacheable memory section. These variables are cleared to zero by start-up code.

7.2 Linker command file

Memory shall be allocated for every section defined in the driver's CanTrcv_43_tja1145a_MemMap.h.



Chapter 8

Integration Steps

This section gives a brief overview of the steps needed for integrating this module:

1. Generate the required module configuration(s). For more details refer to section [Files Required for Compilation](#)
2. Allocate the proper memory sections in the driver's memory map header file (CanTrcv_43_tja1145a_MemMap.h) and linker command file. For more details refer to section [Sections to be defined in CanTrcv_43_tja1145a_MemMap.h](#)
3. Compile & build the module with all the dependent modules. For more details refer to section [Building the Driver](#)



Chapter 9

External assumptions for driver

None.

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

